Lecture 26.05: Linear Regression with Gradient Descent — Step 3: Analytical Gradient Derivation

📌 Executive Summary

Rigorous first-principles calculus derivation of the cost function partial derivatives with respect to weights and bias, dimension verification, and vectorized gradient expressions.

🔬 Mathematical Formulation & Theory

1. The Multivariable Calculus Chain Rule

We seek the gradient vector ∇J, which indicates the direction of steepest ascent on the cost surface. To minimize J, we must compute:

$$\frac{\partial J}{\partial \mathbf{w}} \quad \text{and} \quad \frac{\partial J}{\partial b}$$

Recall the definition of the cost function:

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

Let $u^{(i)} = \hat{y}^{(i)} - y^{(i)} = \sum_{j=1}^{d} w_j x_j^{(i)} + b - y^{(i)}$. By the chain rule:

$$\frac{\partial J}{\partial w_j} = \frac{1}{2m} \sum_{i=1}^{m} \frac{\partial (u^{(i)})^2}{\partial u^{(i)}} \cdot \frac{\partial u^{(i)}}{\partial w_j}$$

2. Differentiating with Respect to Weights w_j

1. Outer derivative: $\frac{\partial}{\partial u}(u^2) = 2u = 2(\hat{y}^{(i)} - y^{(i)})$
2. Inner derivative: $\frac{\partial}{\partial w_j} \left(\sum_{k=1}^{d} w_k x_k^{(i)} + b - y^{(i)} \right) = x_j^{(i)}$

Multiplying together:

$$\frac{\partial J}{\partial w_j} = \frac{1}{2m} \sum_{i=1}^{m} 2 \left(\hat{y}^{(i)} - y^{(i)} \right) x_j^{(i)} = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

3. Differentiating with Respect to Bias b

1. Outer derivative: $2(\hat{y}^{(i)} - y^{(i)})$
2. Inner derivative: $\frac{\partial}{\partial b}(\cdots + b - y^{(i)}) = 1$

$$\frac{\partial J}{\partial b} = \frac{1}{2m} \sum_{i=1}^{m} 2 \left(\hat{y}^{(i)} - y^{(i)} \right) \cdot 1 = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)$$

4. Vectorized Matrix Formulation

Across all d features simultaneously:

$$\nabla_{\mathbf{w}} J = \frac{1}{m} X^T (\hat{\mathbf{y}} - \mathbf{y})$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})$$

Dimensionality Audit:

- $X \in \mathbb{R}^{m \times d} \implies X^T \in \mathbb{R}^{d \times m}$
- $(\hat{\mathbf{y}} - \mathbf{y}) \in \mathbb{R}^{m \times 1}$
- Product: $X^T (\hat{\mathbf{y}} - \mathbf{y}) \in \mathbb{R}^{d \times 1}$, exactly matching the dimension of weight vector $\mathbf{w}$!

💻 Python & NumPy Implementation

```
import numpy as np

def compute_gradients(X, y, weights, bias):
    """Vectorized computation of analytical gradients dw and db."""
    m = len(y)
```

```
    y_pred = X @ weights + bias
    residuals = y_pred - y

    # Analytical gradients
    db = (1.0 / m) * np.sum(residuals)
    dw = (1.0 / m) * (X.T @ residuals)

    return dw, db

# Verification on toy data
X = np.array([[1.0], [2.0], [3.0]])
y = np.array([2.0, 4.0, 6.0])
w = np.array([0.0])
b = 0.0

dw, db = compute_gradients(X, y, w, b)
print(f"Analytical dw: {dw}, db: {db}")
```

🎯 Key Takeaways & Best Practices

1. **First-Principles Transparency**: Building from scratch demystifies optimizer mechanics, memory layouts, and algorithmic failure modes.
2. **Vectorization vs Loops**: Always eliminate Python `for` loops across sample dimensions by relying on BLAS-accelerated NumPy array operations.
3. **Shape Integrity**: Maintain explicit assertions on input array ranks and shapes (`X.shape == (m, d)` , `y.shape == (m,)`) to prevent silent broadcasting bugs.